API Guidelines

Version: 1.0

Purpose

This document defines the design principles for all internal and external APIs of Trading Platform Pro.
The objective is to provide APIs that are consistent, maintainable, extensible and easy to use.

Scope

These guidelines apply to:
• Internal Python APIs
• Service Interfaces
• Repository Interfaces
• Plugin APIs
• Future REST APIs
• Future WebSocket APIs

API Design Principles

Every API should be:
• simple
• explicit
• predictable
• strongly typed
• well documented
• backwards compatible whenever practical

Naming

Classes
```
OrderService
```
Interfaces
```
OrderRepository
```
Methods
```
load_order()
create_position()
calculate_risk()
```
Avoid abbreviations.

Parameters

Prefer:
```python
create_order(order: Order) -> OrderId
```
Avoid long parameter lists.

Use Value Objects or DTOs where appropriate.

Return Values

Return meaningful objects.
Avoid:
```
True
False
None
```
when richer domain information is available.

Exceptions

APIs should:
• fail explicitly
• provide meaningful exceptions
• avoid silent failures
Document expected exceptions.

Versioning

Public APIs should evolve carefully.
Breaking changes require:
• documentation updates
• migration guidance
• version increment

Dependency Rules

APIs should expose abstractions rather than implementations.
Example:
```
OrderRepository
```
instead of
```
SQLiteOrderRepository
```

Documentation

Every public API should document:
• purpose
• parameters
• return value
• exceptions
• usage examples (where useful)

Testing

Public APIs require automated tests.
Tests should verify:
• expected behaviour

• edge cases
• invalid input
• error handling

API Security

APIs shall:
• validate all input
• reject invalid requests explicitly
• never expose sensitive information
• use least-privilege principles
Security requirements apply to internal and external APIs.

API Review Checklist

Before introducing or changing an API verify:
• interface remains minimal
• naming is consistent
• backward compatibility considered
• documentation updated
• automated tests added
• architecture preserved

Future API Evolution

Future APIs should remain consistent across:
• Python interfaces
• Plugin interfaces
• REST APIs
• WebSocket APIs
Reuse existing domain models and avoid duplicate contracts.

Related Documents

• Architecture.md
• Coding_Standards.md
• Development_Guidelines.md
• AGENTS.md